

Frontend API Guide — Procurement Suite (BOM · Procurement · Inventory · Supplier-PO)

Audience: Frontend developer building the Stage 1–5 procurement screens. **Purpose:** Every endpoint you need to design and wire the UI — method, path, auth, request body, sample response, error codes, status vocabularies, and the flow that ties the five stages together. **Source of truth:** the four modules' `router.py` / `schemas.py` / `presenters.py` / `enums.py`. Sample responses below are transcribed verbatim from the backend presenter functions, so the field names match the live API exactly.

0. Conventions (read this first)

0.1 Base URL & auth

- **All routes** in this guide live under one prefix: `/api/v1/procurement` (the four backend modules each mount `prefix="/procurement"`, so they share it).
- **Auth:** standard OAuth2 password-flow JWT. Send `Authorization: Bearer <token>` on **every** call **except** the supplier-PO tracking pixels and webhooks (Section 4.5), which are deliberately unauthenticated and carry an opaque token instead.
- Login is the platform-wide `POST /api/v1/auth/login` (phone in the `username` field). Not covered here — see `FRONTEND_HANDOFF.md`.

0.2 Roles

Code in this doc	UserRole value	Notes
DM	DIRECT_MANAGER	Superuser — bypasses most role gates.
MD	MANAGING_DIRECTOR	Superuser and the sole BOM approver/rejecter/exporter.
Cutting	CUTTING_MANAGER	Owns the BOM edit + cutting-confirm gate; a PO approver for leather.
Viewer	VIEWER	Read-only (inventory + PO + supplier + board reads).
HR	HR	A PO approver candidate for accessory lines.

Each endpoint below lists its gate. "DM/MD" means either role (MD also passes any DM-gated route as superuser).

0.3 Optimistic concurrency (BOM edits and PO edits share this)

Both `PATCH /boms/{id}/items` and `PATCH /pos/{id}/items` use the same contract:

- You send `base_revision` = the `revision` you last read.
- If someone else edited in between → **409** with `stale_revision` — refetch and retry.
- On success the response carries the **new revision**. Store it for the next edit.
- A PO that has already been sent rejects edits with **409 po_locked**.

0.4 File uploads

- All upload routes take a **multipart** body with a single `file` field.
- Files over `MAX_UPLOAD_MB` are rejected with **413** (streamed + capped server-side).

0.5 Error shape

- Stage-1 upload validation failures return a **rich JSON envelope** (Section 1.4), not a plain `{detail}`. Everything else uses FastAPI's standard `{"detail": "..."} with the HTTP status noted per endpoint.`

1. Procurement — Stage 1: Intake (upload & validation)

What it does: A "submission" is one upload batch with **two slots** — an *order sheet* and a *spec sheet*. Each upload is classified + validated (is this really an order sheet? does it belong to the right client?). When both slots are accepted and virus-clean, the submission is `ready_for_stage_2`.

Screens implied: a two-slot uploader with per-file validation feedback, and a "ready to advance" gate banner.

1.1 Endpoints

Method	Path	Gate	Purpose
POST	<code>/submissions</code>	DM/MD	Open an empty upload batch. → 201
POST	<code>/submissions/{id}/order-sheet</code>	DM/MD	Upload + validate the order slot.
POST	<code>/submissions/{id}/spec-sheet</code>	DM/MD	Upload + validate the spec slot.
GET	<code>/submissions/{id}</code>	DM/MD	Status + the Stage-2 readiness gate.
GET	<code>/submissions/{id}/documents/{document_id}</code>	DM/MD	Full per-document validation report.

1.2 Open a submission

POST /submissions — body is optional; `client_id` may be unknown at upload time (it's derived from the order sheet later).

```
// Request (optional)
{ "client_id": "0f8e...-uuid" } // or send no body

// Response 201
{ "submission_id": "a1b2...-uuid", "status": "open" }
```

1.3 Upload a slot (success)

POST /submissions/{id}/order-sheet (and /spec-sheet) — multipart file. Success is **201** with this envelope:

```
{
  "submission_id": "a1b2...-uuid",
  "document": {
    "id": "d0c0...-uuid",
    "kind": "order_sheet",
    "filename": "KJ_order.xlsx",
    "mime": "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
    "sha256": "...",
    "size_bytes": 48213,
    "page_count": null,
    "storage_url": "...",
    "validation": {
      "status": "accepted",
      "classified_as": "order_sheet",
      "spec_type": null,
      "client_match": "KJ",
      "confidence": 0.97,
      "method": "heuristic",
      "signals_matched": ["po_number", "style_grid"],
      "signals_expected": ["po_number", "style_grid", "size_breakdown"],
      "signals_found": ["po_number", "style_grid", "size_breakdown"],
      "reason_code": null,
      "suggested_fix": null
    },
    "scan_status": "clean"
  },
  "submission": {
    "order_sheet": { "present": true, "validation_status": "accepted" },
    "spec_sheet": { "present": false, "validation_status": null },
    "complete": false,
    "ready_for_stage_2": false,
    "blocking": ["spec_sheet missing"]
  }
}
```

1.4 Upload a slot (rejection)

A file that classifies as the wrong kind/slot, or fails a pre-validation gate, returns the **rejection envelope** with a status from the table in 1.6. Render `validation.suggested_fix` to the user.

```
// e.g. 422 — a spec sheet was dropped into the order slot
{
  "error": "document_validation_failed",
  "submission_id": "a1b2...-uuid",
  "document_fingerprint": {
```

```

    "filename": "spec.pdf", "mime": "application/pdf",
    "sha256": "...", "size_bytes": 91022
  },
  "validation": {
    "status": "rejected",
    "reason_code": "wrong_slot",
    "expected_kind": "order_sheet",
    "confidence": 0.88,
    "signals_expected": ["po_number", "style_grid"],
    "signals_found": ["measurement_table", "grading_increments"],
    "closest_client_profile": "KJ",
    "method": "llm",
    "suggested_fix": "This looks like a spec sheet. Upload it to the spec-sheet
slot instead."
  }
}

```

Re-uploading the **same** rejected file replays the cached verdict (no second validation bill) — same envelope shape.

1.5 Submission status (the gate)

GET /submissions/{id} →

```

{
  "order_sheet": { "present": true, "validation_status": "accepted" },
  "spec_sheet": { "present": true, "validation_status": "accepted" },
  "complete": true,
  "ready_for_stage_2": true,      // ← enable "Advance to BOM" only when true
  "blocking": []                // human-readable reasons while not ready
}

```

GET /submissions/{id}/documents/{document_id} returns the single document block shown in 1.3 (full per-document report).

1.6 Vocabularies

- **SubmissionStatus:** open → complete → consumed (Stage 2 picked it up) | rejected.
- **ValidationStatus:** pending · accepted · rejected · superseded · needs_manual_review.
- **ScanStatus:** clean · skipped · infected · error (clean + skipped pass the gate).
- **ClassificationMethod:** heuristic · llm · manual.
- **RejectReason** → **HTTP status** (map these to user messages):

reason_code	HTTP	Meaning
unsupported_mime	415	Not an accepted file type.
file_too_large	413	Over the upload size cap.
empty_or_corrupt	422	Unreadable file.
virus_detected	422	Scanner flagged it.
scanner_unavailable	503	Retry later.

reason_code	HTTP	Meaning
not_an_order_sheet	422	Wrong document type for the slot.
not_a_spec_sheet	422	Wrong document type for the slot.
wrong_slot	422	Right kind, wrong slot.
needs_manual_review	422	Low confidence — human must decide.
submission_locked	409	Batch already consumed/closed.

2. BOM — Stage 2/3: Editable BOM tree, approval lifecycle, notifications

What it does: Each style gets a Bill of Materials (the cost tree). The Cutting Manager edits material lines (notably the **DCM** = leather area per garment) and confirms cutting; the MD then approves/rejects/exports. Approval is gated on cutting-confirm, and approving spawns an inventory check (Stage 4).

Screens implied: an editable BOM grid (inline cell edits with optimistic revision lock), a cutting-confirm action, an MD review panel (approve/reject/reopen/ export), and a notification bell + toast (a new BOM-ready notice escalates by email after 2h if unopened).

2.1 Endpoints

Method	Path	Gate	Purpose
GET	/boms/{id}	Cutting	The editable BOM tree.
PATCH	/boms/{id}/items	Cutting	Bulk inline edit (optimistic revision lock).
POST	/boms/{id}/confirm-cutting	Cutting	The cutting-manager gate (must precede approval).
POST	/boms/{id}/approve	MD	Approve (optionally lock). Spawns an inventory check.
POST	/boms/{id}/reject	MD	Reject with mandatory reason.
POST	/boms/{id}/reopen	DM/MD	Reopen a rejected/approved BOM for editing.
POST	/boms/{id}/export	MD	Export the approved BOM document.
GET	/notifications	any	The caller's

Method	Path	Gate	Purpose
			notifications (?unread_only=true).
GET	/notifications/stream	any	SSE stream for live toasts.
POST	/notifications/{id}/open	any	Mark opened (stops the email escalation).

2.2 BOM tree response (GET /boms/{id})

```
{
  "id": "b0...-uuid",
  "status": "draft",
  "revision": 3,                                // ← send this as base_revision on the
next PATCH
  "currency": "INR",
  "order_qty": 500,
  "garment_fob_price": 1240.50,
  "bulk_total": 620250.00,
  "cutting_confirmed_at": null,                  // ISO when confirmed; null gates approval
  "approved_at": null,
  "rejection_reason": null,
  "export_document_id": null,
  "exported_at": null,
  "items": [
    {
      "id": "i1...-uuid",
      "category": "main_material",
      "name": "Cow Nappa Leather",
      "material_color": "Black",
      "qty_per_garment": 14.5,                    // the DCM for material lines (dm²)
      "uom": "dm2",
      "unit_price": 32.0,
      "bulk_qty": 7250.0,
      "total_cost": 232000.0,
      "dcm_source": "template",                  // template | similar_style | ai_estimate
| manual
      "dcm_confidence": 0.9,
      "annotation": null
    }
    // ... more lines
  ]
}
```

2.3 Bulk edit (PATCH /boms/{id}/items)

```
// Request – BomBulkPatch. field ∈ {dcm, qty_per_garment, unit_price}
{
  "base_revision": 3,
  "edits": [
    { "bom_item_id": "i1...-uuid", "field": "dcm", "value": 15.0 },
    { "bom_item_id": "i2...-uuid", "field": "unit_price", "value": 28.5 }
  ]
}
```

```
// Response
{
  "revision": 4,                      // store this
  "recomputed": { /* the full BOM tree, same shape as 2.2 */ },
  "reconfirm_required": true          // a DCM edit after cutting-confirm reopens
the §10 gate
}
```

A stale `base_revision` → **409 stale_revision**. A dcm edit is the human source-of-truth (`dcm_source` becomes `manual`); if cutting was already confirmed it resets the gate and `reconfirm_required` is `true`.

2.4 Lifecycle actions

```
// POST /confirm-cutting →
{ "status": "ready_for_review", "templates_backfilled": 2, /* ... */ }

// POST /approve   body (optional): { "lock": true } →
{ "bom_id": "b0...", "status": "approved", "inventory_check_id": "c9...-uuid" }

// POST /reject    body (required): { "reason": "DCM for lining looks doubled" }
→
{ "bom_id": "b0...", "status": "rejected", "rejection_reason": "DCM for lining
looks doubled" }

// POST /reopen   →
{ "bom_id": "b0...", "status": "draft", "revision": 5 }

// POST /export   →
{ "bom_id": "b0...", "status": "exported", "export_document_id": "e1...-uuid",
"sha256": "..." }
```

`approve` is refused (409/422) while `cutting_confirmed_at` is null.

2.5 Notifications

- `GET /notifications?unread_only=true` → array of the caller's notification rows.
- `GET /notifications/stream` → **text/event-stream** (SSE). Subscribe for live badge/toast updates; don't poll.
- `POST /notifications/{id}/open` → stamps `opened_at` and cancels the 2-hour email escalation. Call it when the user opens the item.

2.6 Vocabularies

- **BomStatus**: `draft` → `ready_for_review` → `approved` | `rejected` → `locked` → `exported`. (`approved`/`locked` are immutable; `rejected` is terminal until `reopen`.)
- **BomItemCategory** (9 line types): `main_material`, `sub_material`, `lining`, `interlining`, `thread`, `accessory`, `manufacturing`, `packaging`, `fob_charge`. → `manufacturing` and `fob_charge` are **excluded** from the inventory check.

- **DcmSource:** `template · similar_style · ai_estimate · manual` (priority order).
- **ExtractionSource:** `deterministic · gemini · groq · manual` (audit trail).

3. Inventory — Stage 4: Master stock + per-BOM availability check

What it does: Maintains a master stock list (synced from a spreadsheet) and runs a per-BOM availability check — every BOM material line is matched against stock and gets a verdict (`sufficient / partial / out_of_stock`) with a shortfall. A grouped dashboard rolls the per-BOM badges up to `client` → `order` → `style`.

Screens implied: a stock-master table with import preview, a per-BOM check result panel (line-by-line with badges + shortfall value), and a portfolio dashboard.

3.1 Endpoints

Method	Path	Gate	Purpose
POST	<code>/inventory/preview</code>	DM/MD	Dry-run a stock-master upload.
POST	<code>/inventory/commit</code>	DM/MD	Idempotent upsert of the stock master.
GET	<code>/inventory/items</code>	DM/MD/Viewer	Paged stock list (? <code>search&limit&offset</code>).
POST	<code>/boms/{id}/inventory-check</code>	DM/MD	Run / re-run the check for a BOM.
GET	<code>/inventory-checks/{id}</code>	DM/MD/Viewer	One check's full result.
GET	<code>/inventory-checks</code>	DM/MD/Viewer	Grouped dashboard (? <code>client_id&order_id</code>).
GET	<code>/boms/{id}/inventory-check</code>	DM/MD/Viewer	The latest check for a BOM.

3.2 Stock import preview/commit

Both take multipart `file` and return the same summary:

```
{
  "raw_count": 312,
  "kept": 298,
  "dropped": 14,
  "warnings": ["row 41: missing UOM, defaulted to 'pcs'"],
  "rows": [
    {
```



```

        "normalized_key": "cow-nappa-black",
        "description": "Cow Nappa Leather Black",
        "uom": "dm2", "qty_on_hand": 5200.0, "rate": 31.5,
        "color": "Black", "lots": ["L-2024-08"]
    }
    // ...
}

```

3.3 Stock list (GET /inventory/items)

```

// each item
{
    "id": "s1...-uuid",
    "description": "Cow Nappa Leather Black",
    "normalized_key": "cow-nappa-black",
    "uom": "dm2", "qty_on_hand": 5200.0, "rate": 31.5,
    "color": "Black", "is_active": true
}

```

3.4 Per-BOM check (POST /boms/{id}/inventory-check, GET /inventory-checks/{id})

```

{
    "inventory_check_id": "c9...-uuid",
    "bom_id": "b0...-uuid",
    "status": "complete", // running | complete
    "run_at": "2026-06-13T09:12:44+00:00",
    "summary": {
        "badge": "partial", // worst line status = the BOM badge
        "lines_total": 12,
        "sufficient": 9, "partial": 2, "out_of_stock": 1,
        "flags": { "unmatched": 1, "uom_mismatch": 0 },
        "shortfall_value": 18450.00,
        "currency": "INR"
    },
    "lines": [
        {
            "bom_item_id": "i1...-uuid",
            "category": "main_material",
            "name": "Cow Nappa Leather", "material_color": "Black",
            "required_qty": 7250.0, "uom": "dm2",
            "matched": {
                "inventory_item_id": "s1...-uuid",
                "description": "Cow Nappa Leather Black",
                "method": "key", // key | alias | manual
                "uom": "dm2"
            },
            "on_hand_qty": 5200.0,
            "available_qty": 5200.0, // on_hand minus other active
            "reservations": {
                "reserved_for_this_bom": 5200.0,
                "shortfall_qty": 2050.0,
                "status": "partial", // sufficient | partial | out_of_stock
                "flags": [], // e.g. ["unmatched"], ["uom_mismatch"]
                "suggestion": null // a non-applied fuzzy match hint, if any
            },
            // ...
        },
        // ...
    ],
    "excluded": [ // manufacturing/fob_charge lines, not

```

```
checked
  { "bom_item_id": "i9...-uuid", "name": "CMT Charge", "category":
    "manufacturing" }
  ]
}
```

GET /boms/{id}/inventory-check returns the **latest** check for that BOM in the same shape (re-rendered from the persisted rows — available_qty falls back to on_hand_qty).

3.5 Dashboard (GET /inventory-checks)

```
{
  "clients": [
    {
      "client_id": "0f...-uuid", "client_name": "KJ",
      "orders": [
        {
          "client_order_id": "o1...-uuid", "order_number": "PO-1182",
          "styles": [
            {
              "style_id": "y1...-uuid", "style_name": "Carnaby Jacket",
              "bom_id": "b0...-uuid", "inventory_check_id": "c9...-uuid",
              "badge": "partial", "shortfall_lines": 3,
              "checked_at": "2026-06-13T09:12:44+00:00"
            }
          ]
        }
      ]
    }
  ],
  "totals": { "boms_checked": 18, "fully_sufficient": 11, "with_shortfall": 7 }
}
```

3.6 Vocabularies

- **InventoryCheckStatus:** running · complete.
- **InventoryLineStatus** (the badge): sufficient · partial · out_of_stock.
- **MatchMethod:** key (exact normalized) · alias (curated synonym) · manual. A fuzzy candidate is only ever a suggestion, never an auto-applied match.
- **ReservationStatus** (background): active (counts against available) · released · consumed.

4. Supplier-PO — Stage 5: Suppliers, PO state machine, tracking, board

What it does: From an approved BOM, generates supplier purchase orders, matches each to a supplier (or flags needs_supplier), runs a PO through a submit → approve → send → acknowledge lifecycle with cross-check approver routing (leather lines → Cutting, accessory lines → MD/DM/HR), tracks email opens/clicks and an escalation ladder, and surfaces an order/style **production board**.

Screens implied: a supplier directory + add/edit form, a PO editor (line grid + GST money block), a PO action bar (the state machine), a per-PO tracking timeline, and a kanban-style production board.

4.1 Suppliers (§9)

Method	Path	Gate	Purpose
POST	/suppliers/import/preview	DM/MD	Dry-run a supplier-list upload.
POST	/suppliers/import/commit	DM/MD	Upsert suppliers.
GET	/suppliers	DM/MD/Viewer	List (?q&service&active&limit&offset).
GET	/suppliers/{id}	DM/MD/Viewer	One supplier (+ history + open POs).
POST	/suppliers	DM/MD	Create. → 201
PATCH	/suppliers/{id}	DM/MD	Update.
DELETE	/suppliers/{id}	MD	Soft-delete (deactivate).
POST	/suppliers/{id}/reactivate	MD	Reactivate.

SupplierCreate / SupplierUpdate body (all optional except name on create):

```
{
  "name": "Acme Leather Co",
  "phone": "9000011111", "email": "sales@acme.example",
  "service": "leather", "gstn": "33ABCDE1234F1Z5",
  "address": "Chennai, TN", "currency": "INR",
  "payment_terms_days": 30, "lead_time_days": 14,
  "supplier_type": "leather", // leather | accessory | service
  "whatsapp_phone": "9000011111"
}
```

Supplier response (supplier_block):

```
{
  "id": "v1...-uuid", "name": "Acme Leather Co",
  "phone": "9000011111", "email": "sales@acme.example",
  "service": "leather", "gstn": "33ABCDE1234F1Z5", "address": "Chennai, TN",
  "currency": "INR", "payment_terms_days": 30, "lead_time_days": 14,
  "is_active": true, "email_status": "valid", // unknown | valid |
invalid
  "supplier_type": "leather", "state_code": "33",
  "whatsapp_phone": "9000011111", "has_contact": true,

  // only on GET /suppliers/{id}:
  "supply_history": [
    { "normalized_description": "cow-nappa-black", "raw_description": "Cow Nappa
Black",
      "mode": "leather", "uom": "dm2", "txn_count": 7,
```

```

    "last_purchased_at": "2026-04-01T00:00:00+00:00",
    "last_rate": 31.0, "min_rate": 29.5, "max_rate": 33.0 }
  ],
  "open_pos": [ { "id": "p1...-uuid", "po_number": "SP0-0042", "status": "sent",
    "total": 232000.0 } ]
}

```

4.2 Purchase orders (§1–§7)

Method	Path	Gate	Purpose
POST	/boms/{id}/generate-pos	DM/MD	Generate PO(s) from an approved BOM.
GET	/pos	DM/MD/Viewer	List (? status&needs_supplier&bom_id&supplier_id).
GET	/pos/{id}	DM/MD/Viewer	One PO (full envelope).
PATCH	/pos/{id}/items	DM/MD	Bulk edit (revision lock, see 0.3).
POST	/pos/{id}/submit	DM/MD	draft → pending_approval.
POST	/pos/{id}/approve	Cutting/MD/DM/HR*	Approve (routed by line type).
POST	/pos/{id}/reject	Cutting/MD/DM/HR*	Reject with reason.
POST	/pos/{id}/send	DM/MD	Send to supplier (locks edits).
POST	/pos/{id}/cancel	DM/MD	Cancel (from any pre-send state).
POST	/pos/{id}/acknowledge	DM/MD	Manually mark confirmed.

* **Approver routing**: the router admits Cutting/MD/DM/HR, but the **service** enforces the real rule — **leather** lines must be approved by **Cutting**, **accessory** lines by **MD/DM/HR**. The UI should show the action to all four roles and surface a 403 if the wrong role tries the wrong PO.

PO view (po_view, returned by every PO endpoint):

```

{
  "id": "p1...-uuid", "po_number": "SP0-0042",
  "status": "sent", "revision": 2,
  "supplier_id": "v1...-uuid", "bom_id": "b0...-uuid", "client_order_id":
  "o1...-uuid",
  "buyer_ref": "P0-1182", "issue_date": "2026-06-13", "delivery_days": 14,
  "payment_terms_days": 30, "currency": "INR", "gst_mode": "intra",
  "subtotal": 196610.17, "cgst": 17694.92, "sgst": 17694.92, "igst": 0.0,
  "round_off": 0.01, "total": 232000.00,
  "needs_supplier": false, "no_contact_channel": false,

```

```

    "match_method": "history", "candidates": [ /* suggested suppliers when
unmatched */ ],
    "approved_at": "2026-06-13T08:00:00+00:00", "rejected_at": null,
    "rejection_reason": null,
    "sent_at": "2026-06-13T08:30:00+00:00",
    "pdf_document_id": "pdf...-uuid",
    "first_opened_at": "2026-06-13T09:01:00+00:00", "first_clicked_at": null,
    "current_rung": 0, "next_escalation_at": "2026-06-13T11:30:00+00:00",
    "acknowledged_at": null, "acknowledged_channel": null,
    "items": [
      {
        "id": "li1...-uuid", "item_no": 1, "description": "Cow Nappa Leather",
        "color": "Black", "uom": "dm2", "qty": 7250.0, "unit_price": 27.12,
        "amount": 196610.0, "inventory_item_id": "s1...-uuid", "bom_item_id":
"i1...-uuid"
      }
    ],
    "supplier": { "id": "v1...-uuid", "name": "Acme Leather Co", "email":
"sales@acme.example",
      "phone": "9000011111", "gstn": "...", "address": "Chennai, TN",
      "supplier_type": "leather", "email_status": "valid" }
  }

```

GET /pos returns { "purchase_orders": [...po_view...], "count": 12 }.

PO request bodies:

```

// PATCH /pos/{id}/items – PoBulkPatch. field ∈ {description, color, uom, qty,
unit_price}
{
  "base_revision": 2,
  "item_edits": [ { "po_item_id": "li1...-uuid", "field": "unit_price",
"value": 27.5 } ],
  "po_edits": { "delivery_days": 10 },
  "add_items": [ { "description": "YKK Zipper", "color": "Black", "uom":
"pcs",
                    "qty": 500, "unit_price": 18.0, "bom_item_id": null,
                    "inventory_item_id": null } ],
  "remove_item_ids": [ "li9...-uuid" ]
}

// POST /pos/{id}/reject
{ "reason": "Rate exceeds last purchase by 20%" }

// POST /pos/{id}/acknowledge – channel records how it was confirmed
{ "channel": "manual", "confirmed_qty": 7250.0, "notes": "Supplier confirmed by
phone" }

```

4.3 Production board (§8)

Method	Path	Gate	Purpose
GET	/production-tracking	DM/MD/Viewer	The board (?client_id&order_id).
POST	/production-tracking/{id}/transition	DM/MD/Cutting	Manual status move.

```
// each board card (tracking_view)
{
  "id": "t1...-uuid",
  "client_order_id": "o1...-uuid", "order_number": "PO-1182", "client_name":
  "KJ",
  "style_id": "y1...-uuid", "style_name": "Carnaby Jacket", "bom_id": "b0...-
  uuid",
  "status": "po_confirmed", "po_count": 3, "po_confirmed_count": 2,
  "material_ready_at": null, "released_at": null
}

// POST .../transition body
{ "status": "released_to_production" }
```

4.4 Vocabularies

- **POStatus:** draft → pending_approval → approved → sent → responded → confirmed; plus escalated, rejected (→ back to draft), cancelled (any pre-send state).
- **SupplierType:** leather · accessory · service (drives PO template + approver routing).
- **SupplierEmailStatus:** unknown · valid · invalid (bounce feedback gates email send).
- **POResponseChannel:** email · whatsapp · call.
- **POResponseStatus:** pending · opened · confirmed · no_response · failed.
- **POTrackingEventType:** open · click · delivery · bounce · complaint · whatsapp · call.
- **ProductionTrackingStatus** (the board ladder): awaiting_bom → bom_approved → inventory_checked → po_raised → po_confirmed → material_ready → released_to_production → in_production → completed.

4.5 Unauthenticated routes — do not call these from the app

These are hit by a supplier's mail client / Twilio / Amazon SES, not by your UI. Listed only so you don't accidentally wire a token onto them or try to render them as app pages.

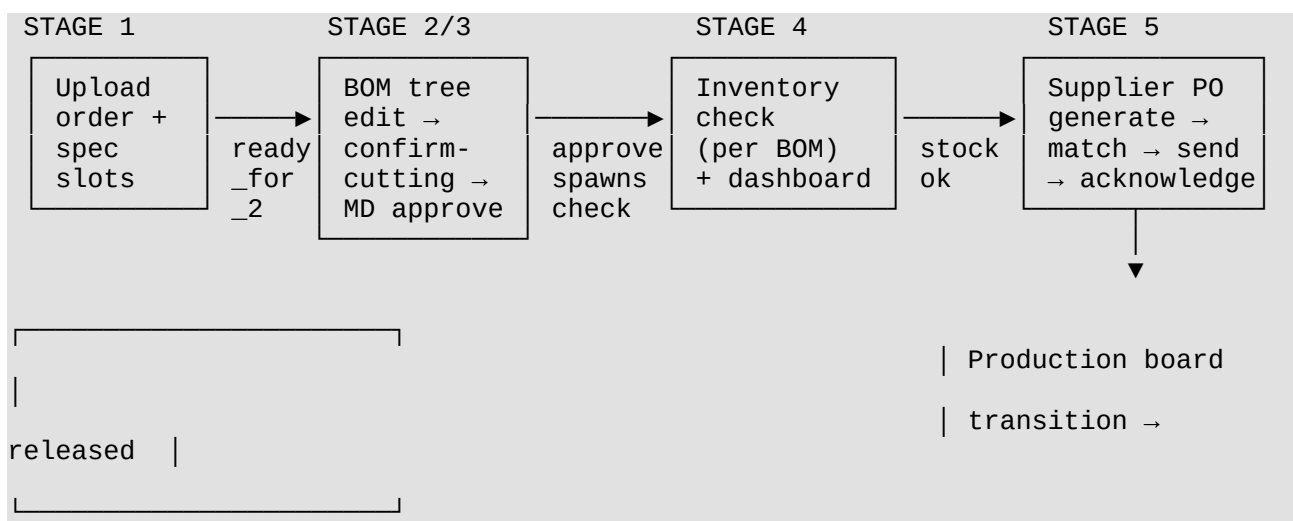
Method	Path	Who calls it
GET	/t/o/{token}.gif	Email client (open-tracking pixel).
GET	/t/c/{token}	Email client (click → 302 redirect).
POST	/webhooks/ses	Amazon SES (bounce/complaint/delivery).
POST	/webhooks/twilio/whatsapp	Twilio (supplier WhatsApp reply).

Method	Path	Who calls it
POST	/webhooks/twilio/voice	Twilio (press-1 voice acknowledge).

The engagement they record surfaces back to your UI through the PO view's `first_opened_at` / `first_clicked_at` / `acknowledged_*` fields (4.2).

5. End-to-end flow (screen sequence)

The five stages chain through shared IDs. A BOM is the spine: a submission produces it, the inventory check and PO generation both hang off `bom_id`, and the production board mirrors the whole thing per style.



1. **Upload** order + spec into a submission; poll `GET /submissions/{id}` until `ready_for_stage_2 = true`.
2. **BOM**: open the tree (`GET /boms/{id}`), edit lines, confirm-cutting, then MD approve — the approve response hands you the `inventory_check_id`.
3. **Inventory**: open that check (`GET /inventory-checks/{id}`) or run/re-run it; the dashboard rolls badges up. Resolve shortfalls before proceeding.
4. **Supplier-PO**: generate - pos from the BOM, attach a supplier (or resolve `needs_supplier`), then walk a PO `submit → approve → send → acknowledge`.
5. **Board**: the order/style advances up `ProductionTrackingStatus`; the human go/no-go is the `released_to_production` transition.

Status-badge legend (for picking UI colours)

Stage	Field	Values (suggested colour intent)
Stage 1	ValidationStatus	accepted · pending ⌚ · needs_manual_review ⚠ · rejected · superseded ○
Stage 2/3	BomStatus	draft ○ · ready_for_review · approved/locked · exported · rejected
Stage 4	InventoryLineStatus/ badge	sufficient · partial · out_of_stock
Stage 5	POStatus	draft ○ · pending_approval · approved ● · sent · confirmed · escalated ● · rejected/cancelled
Stage 5	ProductionTrackingStatus	a left → right ladder (awaiting_bom ... completed) — render as a stepper.

Generated from the live backend modules `bom`, `procurement`, `inventory`, `supplier_po`. If a field here disagrees with the API, the API wins — regenerate `openapi.json` (`python -m scripts.export_openapi`) and reconcile.